

Arquitectura de Microservicios con Contenedores

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Del Monolito a Microservicios

Temario

1. ¿Por qué (y si) migrar?
2. Encontrar los seams
3. Patrones de migración incremental
4. Separar los datos
5. Organización y operación
6. Taller grupal
7. Cierre

Objetivos de la clase

Al terminar, deberían poder:

1. **Juzgar** si una migración tiene sentido, y reconocer los anti-patterns (monolito distribuido, "microservicios porque Netflix").
2. **Encontrar** servicios candidatos usando DDD y datos del sistema en producción.
3. **Aplicar** patrones incrementales (Strangler Fig, Branch by Abstraction, Parallel Run) en vez de un rewrite big-bang.
4. **Planificar** cómo partir una base de datos compartida – la parte más difícil.
5. **Armar** un roadmap de migración por fases, con riesgos y métricas de éxito.

¿De dónde venimos?

Esta clase no introduce herramientas nuevas: **junta** las que ya vieron y las ordena en el tiempo.

Ya lo vieron en...	Hoy lo usamos para...
domain-driven-design	Encontrar dónde cortar
comunicacion, async-patterns-kafka	Hablar entre el monolito y los servicios nuevos
api-gateway	Interceptar y redirigir tráfico (Strangler Fig)
estado-distribuido-patterns, sistemas-distribuidos	Sagas, consistencia eventual, Outbox
testing, devops	Los prerequisites para migrar sin romper
data-plane	Operar lo que queda después

¿Por qué (y si) migrar?

Tres historias reales

Empresa	Qué hizo	Por qué
Amazon (~2001-2006)	Del monolito <i>Obidos</i> a servicios con APIs, equipos chicos dueños de cada uno	El monolito frenaba a cientos de desarrolladores que necesitaban desplegar por separado
Segment (2018)	Volvió de ~140 microservicios a un monolito para sus <i>destinations</i>	Tres ingenieros pasaban la mayor parte del tiempo manteniendo servicios, no agregando valor
Prime Video (2023)	Pasó su servicio de monitoreo de calidad de video de funciones serverless distribuidas a un solo proceso	Costo de orquestación y transferencia de datos entre pasos; bajó ~90% el costo

Moraleja: migrar no es "progreso". Es una decisión con costos, y a veces la decisión correcta es la inversa.

¿Para qué migrar? Razones que sí alcanzan

- **Despliegue independiente:** un equipo sale a producción sin coordinar con otros cinco.
- **Autonomía de equipos:** cada equipo es dueño de su servicio, de punta a punta (ley de Conway).
- **Escalado selectivo:** el catálogo recibe 100× más tráfico que la facturación.
- **Aislamiento de fallas:** si se cae el recomendador, la gente sigue comprando.
- **Stack distinto donde hace falta:** un motor de búsqueda, un modelo de ML.

Microservices are not the goal. They are a means to an end.

– Sam Newman, *Monolith to Microservices*

Pregunta de control: ¿qué problema del negocio resuelve esto, y cómo voy a medir que se resolvió?

Razones que no alcanzan

- **Hype:** "Netflix lo hace". Netflix tiene miles de ingenieros y un problema de escala que casi nadie tiene.
- **"El código es feo":** un rewrite no arregla la falta de tests ni de diseño. Distribuirlo lo hace **más** difícil de arreglar.
- **Performance:** una llamada en memoria cuesta nanosegundos; una por red, milisegundos. Microservicios rara vez hacen algo *más rápido*.
- **"Para escalar":** un monolito se escala horizontal detrás de un load balancer. Hace falta mostrar que *una parte* escala distinto que el resto.
- **CV-driven development:** querer usar Kubernetes y Kafka no es un requerimiento del negocio.

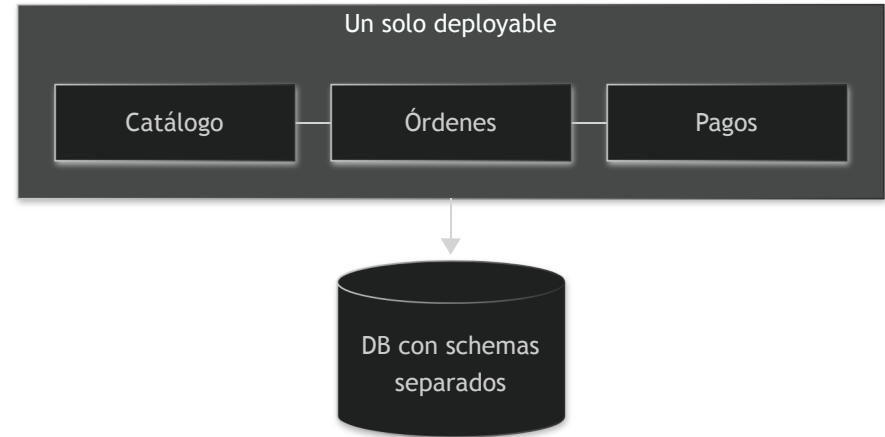
La alternativa: el monolito modular

Un solo deployable, pero con **módulos con fronteras explícitas**:

- Cada módulo = un bounded context, con su API pública.
- Prohibido llamar internals de otro módulo (se valida en CI).
- Idealmente, cada módulo con su **propio schema** en la DB.

Te da gran parte de los beneficios de diseño **sin** pagar la red, el deploy distribuido ni la consistencia eventual.

Es un **paso intermedio** excelente... y muchas veces, el **destino final**. Shopify es el ejemplo más conocido.



Prerrequisitos: "you must be this tall"

Antes de extraer el primer servicio, tiene que existir:

- **CI/CD:** desplegar un servicio tiene que ser trivial y repetible (deck devops).
- **Observabilidad básica:** logs centralizados, métricas y un `correlationId` que cruce servicios.
- **Tests automatizados:** sobre todo tests de caracterización del monolito y contract tests (deck testing).
- **Provisioning rápido:** levantar un servicio nuevo en horas, no en semanas.

Sin esto, cada servicio nuevo multiplica el trabajo manual y los incidentes.

Encuesta rápida: ¿migrarían?

Levanten la mano – **sí**, **no**, o **todavía no**.

1. **Startup de 6 devs**, producto de 8 meses, todavía buscando product-market fit. El código "ya está feo".
2. **E-commerce con 80 devs en 9 equipos**. Un release por mes porque hay que coordinar a todos. Black Friday tira abajo el sitio entero porque el checkout comparte proceso con el catálogo.
3. **Sistema de liquidación de sueldos** de una empresa, 3 devs, carga estable, cambia dos veces por año por regulación.

Encontrar los seams

Repaso: DDD como mapa

Lo vieron en domain-driven-design. Hoy lo aplicamos a un sistema **que ya existe**.

- **Bounded context:** el mejor candidato a frontera de servicio.
- **Ubiquitous language:** si la misma palabra (Cliente, Producto) significa cosas distintas en dos partes del código, ahí hay una frontera.
- **Context mapping:** cómo se relacionan los contextos (customer/supplier, conformist, ACL). Nos dice qué integración vamos a necesitar al separar.

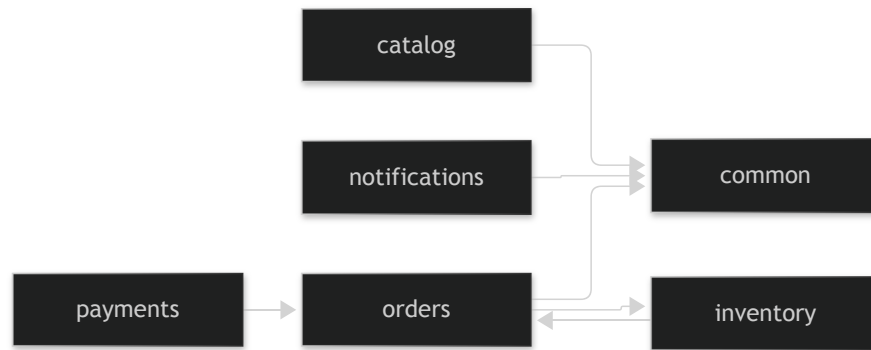
La diferencia con un proyecto nuevo: el código existente tiene fronteras *implícitas*, casi siempre violadas. Hay que descubrirlas en el código, no solo en la pizarra.

Seam #1: dependencias en el código

Analizar el grafo de dependencias entre paquetes o módulos:

- **Alta cohesión adentro, bajo acoplamiento afuera:** eso es un buen candidato.
- **Ciclos** entre módulos: hay que romperlos *antes* de extraer.
- **"God modules"** (utils, common, core) de los que depende todo: suelen esconder lógica de dominio mezclada.

Herramientas: jdeps, dependency-cruiser, Structure101, Sonargraph, o el IDE.



notifications solo depende de common: buen candidato. orders ↔ inventory tienen un ciclo: hay que romperlo primero.

Seam #2: change coupling en git

El código dice qué *depende* de qué. El historial dice qué **cambia junto**.

- Archivos que aparecen juntos en muchos commits están acoplados, aunque no se importen entre sí.
- Si `Order.java` y `InventoryService.java` cambian juntos en el 70% de los commits, separarlos en dos servicios significa **deploys coordinados** para siempre.

Herramienta: *code-maat* (Adam Tornhill, *Your Code as a Crime Scene*), o CodeScene.

```
change coupling a mano

# (commit, archivo): base para contar pares
git log --since="1 year ago" \
  --name-only --format="--" \
  | awk '/^--$/ {n++; next} NF {print n, $0}' \
  | sort -u > commits.txt

# Top de archivos más cambiados (hotspots)
git log --since="1 year ago" \
  --name-only --format="" \
  | sort | uniq -c | sort -rn | head
```

Seams #3 y #4: producción y personas

Datos de runtime

- ¿Qué módulos tienen **carga distinta**? (el catálogo en Black Friday vs. la facturación)
- ¿Cuáles tienen requisitos distintos de **disponibilidad** o **latencia**?
- ¿Qué endpoints comparten tablas "calientes"? (APM, slow query log)

Organización

- ¿Qué equipo es **dueño** de cada área? ¿Dónde se pisan dos equipos?
- ¿Con qué **frecuencia** se cambia cada área? Lo que cambia seguido es lo que más se beneficia de un deploy independiente.
- ¿Qué parte cambia **por razones distintas** (regulación, marketing, operaciones)?

¿Cuál es el primer servicio?

Buscamos: **bajo acoplamiento, valor visible, riesgo bajo.**

- **Buenos primeros candidatos:** notificaciones, catálogo (mucho read), búsqueda, auth (si ya es un módulo aparte).
- **Malos primeros candidatos:** pagos, órdenes – centrales, muy acoplados y caros si fallan.

El primer servicio no es para ganar mucho: es para **aprender el camino** (pipeline, observabilidad, rollback) con poco en juego.

Poco acoplado · alto valor

Empezá por acá

Muy acoplado · alto valor

Más adelante, con experiencia

Poco acoplado · bajo valor

Buen "piloto" de práctica

Muy acoplado · bajo valor

Dejalo en el monolito

Eje horizontal: facilidad de extracción · eje vertical: beneficio

Granularidad: cuidado con los nano-servicios

Un servicio **demasiado chico** es tan problemático como uno demasiado grande.

- **Síntoma:** para resolver un caso de uso hacen falta 6 llamadas por red.
- **Síntoma:** dos servicios siempre se despliegan juntos.
- **Síntoma:** un servicio por entidad (UserService, AddressService, PhoneService).

Heurística: un servicio por bounded context (o por subdominio dentro de uno grande). Empezar **más grande** y partir después es mucho más barato que juntar.

Juntar dos servicios implica migrar datos y reescribir contratos. Partir un módulo bien encapsulado, no.

Antes de extraer: tres puertas (gates)

Encontrar el acoplamiento no es lo mismo que tener un seam que se pueda cortar. Antes de sacar la **tajada** (la funcionalidad a extraer) del proceso:

1. **Tests de caracterización** sobre el borde público de esa tajada. El corte se permite cuando esos tests siguen verdes (deck testing).
2. **Reorganizar adentro del monolito:** el código pasa detrás de una interfaz local y el resto del sistema llama a esa interfaz. El deployable no cambió. (Es el primer paso de Branch by Abstraction, que viene enseguida.)
3. **Inventario de efectos laterales:** lo que el grafo de paquetes **no** muestra – crons, listeners, mailers, archivos, procesos batch enganchados al módulo.

Si un job nocturno o un listener se queda en el monolito, el servicio nuevo es **media capacidad**: el monolito sigue escribiendo "sus" datos por atrás.

 En Notificaciones: tests sobre `send()` y los templates · todo detrás de una interfaz `Notifier` · y el inventario encuentra el cron de reintentos y el mail del resumen semanal.

Patrones de migración incremental

Big-bang rewrite vs. incremental

Big-bang rewrite: "lo hacemos de nuevo, bien, y un día cambiamos".

- El sistema viejo **sigue cambiando** mientras se reescribe: target móvil.
- **Efecto segundo sistema:** el rewrite quiere arreglar *todo* y crece sin límite.
- Cero valor entregado hasta el día del corte, que casi siempre se atrasa.
- El conocimiento escondido en el código viejo (bugs que son features) se pierde.

Incremental: extraer de a una funcionalidad, en producción, con posibilidad de volver atrás.

- Valor entregado **en cada paso**.
- Cada paso es chico, **reversible** y se puede frenar.
- El monolito y los servicios **conviven** durante meses (o años).

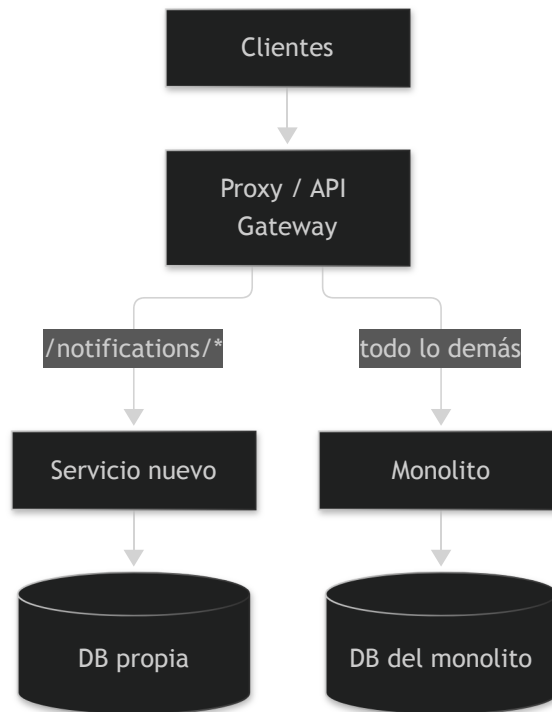
Si el paso no se puede revertir, es demasiado grande.

Strangler Fig

Nombre tomado de una planta que crece alrededor de un árbol hasta reemplazarlo.

1. Poner una **capa de interceptación** delante del monolito (proxy, API Gateway).
2. Implementar una funcionalidad en un **servicio nuevo**.
3. Redirigir **esa ruta** al servicio nuevo. El resto sigue yendo al monolito.
4. Repetir. El monolito se achica.

El cliente no se entera: la URL es la misma.



Strangler Fig en la práctica

La capa de interceptación puede ser tan simple como un `location` de nginx.

- **Rollback:** volver a apuntar la ruta al monolito. Es un cambio de config, no de código.
- **Gradual:** con un gateway (Traefik, Kong, Envoy) se puede mandar el 5% del tráfico primero.
- Lo mismo aplica para la UI: *UI composition* (micro-frontends) reemplaza de a una pantalla.
- La fachada es una **tabla de rutas temporal**: reenvía, registra, permite volver. Si empieza a acumular reglas de negocio, es un **segundo monolito**.

nginx.conf

```
upstream monolith      { server monolith:8080; }
upstream notifications { server notifications:8000; }

server {
    listen 80;

    # Ya extraído → servicio nuevo
    location /api/notifications/ {
        proxy_pass http://notifications;
    }

    # Todo lo demás → monolito
    location / {
        proxy_pass http://monolith;
    }
}
```

Branch by Abstraction

Para funcionalidad que se usa **desde adentro** del monolito (no hay ruta que interceptar).

1. Crear una **abstracción** (interfaz) delante de la funcionalidad actual.
2. Hacer que todo el monolito use la abstracción.
3. Escribir una **segunda implementación** que llama al servicio nuevo.
4. Cambiar de implementación (con un toggle).
5. Borrar la implementación vieja... y, si no aporta, la abstracción.

Todo en `main`, sin branches largos: el código nuevo está desplegado pero apagado.

 La ruta HTTP ya la movió el Strangler; esto cubre lo que el checkout y el cron llaman **desde adentro**.

```
branch-by-abstraction.py

class Notifier(Protocol):
    def send(self, user_id: str, msg: str) → None: ...

class InProcessNotifier:          # código viejo, movido acá
    def send(self, user_id, msg):
        smtp.send(lookup_email(user_id), msg)

class NotificationServiceClient: # nuevo
    def send(self, user_id, msg):
        http.post(f"{NOTIF_URL}/notifications",
                  json={"userId": user_id, "msg": msg})

def get_notifier() → Notifier:
    if flags.enabled("notifications-service"):
        return NotificationServiceClient()
    return InProcessNotifier()
```

Parallel Run y Dark Launching

Cuando equivocarse es caro (precios, impuestos, cálculo de riesgo), no basta con cambiar y rezar.

- **Parallel Run:** cada request va a la implementación **vieja y nueva**. Se devuelve el resultado de la **vieja**; se **comparan** y se registran las diferencias.
- **Dark launching:** el servicio nuevo recibe tráfico real pero su respuesta **no llega** al usuario (tráfico espejado).
- Se cambia de implementación recién cuando las diferencias son cero (o explicadas).

Cuidado con los efectos secundarios:

- Shadowing de una **lectura** (catálogo, una cotización) es seguro.
- Shadowing de una **escritura** (mails, pagos, reservas de stock) manda, cobra o reserva dos veces: apagar el efecto en la sombra (stubs) o usar **clave de idempotencia** (deck estado-distribuido-patterns).

Decorating Collaborator y CDC

Para **agregar** comportamiento nuevo sin tocar el monolito.

Decorating Collaborator

- Un proxy deja pasar la llamada al monolito y, **con la respuesta**, dispara algo en un servicio nuevo.
- Ej.: después de cada POST /orders exitoso, el servicio de *loyalty* suma puntos.

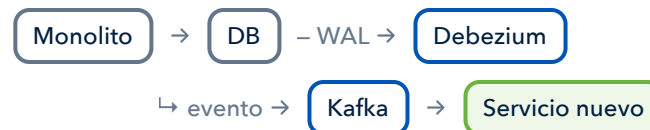
Change Data Capture (CDC)

- Se escucha el **log de cambios de la DB** del monolito (Debezium → Kafka).
- Cada INSERT en orders se convierte en un evento que consume el servicio nuevo.

Decorating Collaborator



Change Data Capture



En los dos casos, el monolito **no se modifica**: lo nuevo (en verde) se engancha desde afuera.

Feature toggles y canary releases

Separar el deploy del release.

- **Feature toggle:** el código nuevo se despliega apagado; se prende por config.
- **Canary release:** el servicio nuevo recibe primero el 1%, después el 10%, el 50%...
- Si las métricas empeoran (errores, latencia), se vuelve al 0%. **Rollback en segundos, sin deploy.**

Trabajo on the fly: un pedido que empezó en el monolito **termina** en el monolito; solo el tráfico nuevo va al servicio. Se drena, no se migran procesos a mitad de camino.

Día 1 – 1% de los mails por el servicio nuevo, solo usuarios internos

Día 3 – 10%, se miran errores, p99 y mails rebotados

Día 7 – 50%

Día 10 – 100%, se borra InProcessNotifier

En cualquier momento – 0% si algo se rompe

Anti-Corruption Layer

Conviviendo con el monolito, el servicio nuevo **hereda su modelo** si no se cuida (flags de 2009, tablas de 80 columnas).

- El **ACL** traduce, en el borde, el modelo del monolito al lenguaje del nuevo contexto.
- Cuando el monolito desaparece, se borra el ACL y el modelo interno queda intacto.

Ojo: si el servicio llama **sincrónicamente** al monolito en cada request, el acoplamiento pasó de la DB a HTTP: un monolito distribuido formándose.

acl.py

```
# Monolito: CUST_MSTR con flags heredados
def to_customer(row: dict) → Customer:
    return Customer(
        id=CustomerId(row["CUST_ID"]),
        email=row["EMAIL_ADDR"].strip().lower(),
        tier=Tier.GOLD if row["FLG_VIP"] == "Y"
        else Tier.STANDARD,
        active=row["STATUS_CD"] not in ("D", "X"),
    )
```

¿Qué patrón uso?

Situación	Patrón
La funcionalidad se accede por una ruta HTTP desde afuera	Strangler Fig
La funcionalidad se usa desde adentro del monolito	Branch by Abstraction
Equivocarse es caro ; necesito comparar resultados	Parallel Run / Dark launching
Quiero agregar algo sin tocar el monolito	Decorating Collaborator, CDC
Quiero pasar tráfico de a poco y poder volver	Feature toggles, canary
El servicio nuevo tiene que hablar con el modelo viejo	Anti-Corruption Layer

No son excluyentes: una migración real usa varios a la vez.



Break – 10 min

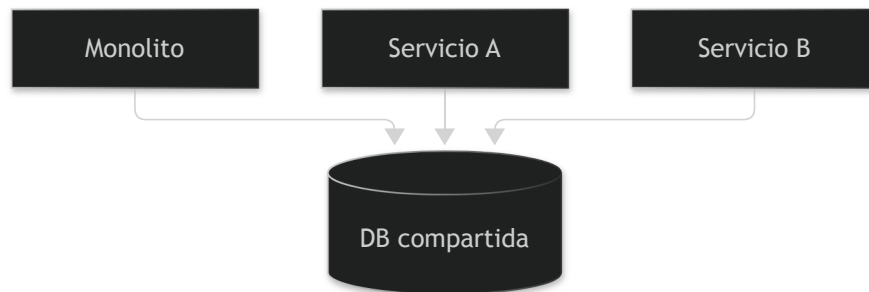
Separar los datos

La DB compartida: el acoplamiento más peligroso

Si el servicio nuevo lee y escribe **las mismas tablas** que el monolito:

- Nadie puede cambiar un schema sin coordinar con todos.
- No hay forma de saber **quién es dueño** de un dato.
- Las reglas de negocio se duplican (o se saltean) al escribir directo en la tabla.
- Un deploy independiente no sirve de nada si el schema no es independiente.

Eso es un **monolito distribuido**: todos los costos de la red, ninguno de los beneficios.



La DB es la API real del sistema, y nadie la diseñó como API.

¿Qué va primero, el código o los datos?

Código primero

- Extraer el servicio, que sigue usando la DB del monolito por un tiempo.
-  Valor rápido, se aprende el pipeline.
-  Riesgo de que "por un tiempo" sea para siempre.
-  No descubrís los problemas de datos hasta el final.

Datos primero

- Separar el schema dentro del monolito; el código sigue en el monolito.
-  Se descubren temprano los joins y transacciones que se rompen.
-  Todavía es reversible y barato.
-  Más tiempo sin un beneficio visible.

¿Dudás de la frontera (joins, transacciones)? Datos primero. ¿Necesitás aprender el camino? Código primero, con fecha para los datos.

Database View y Database Wrapping Service

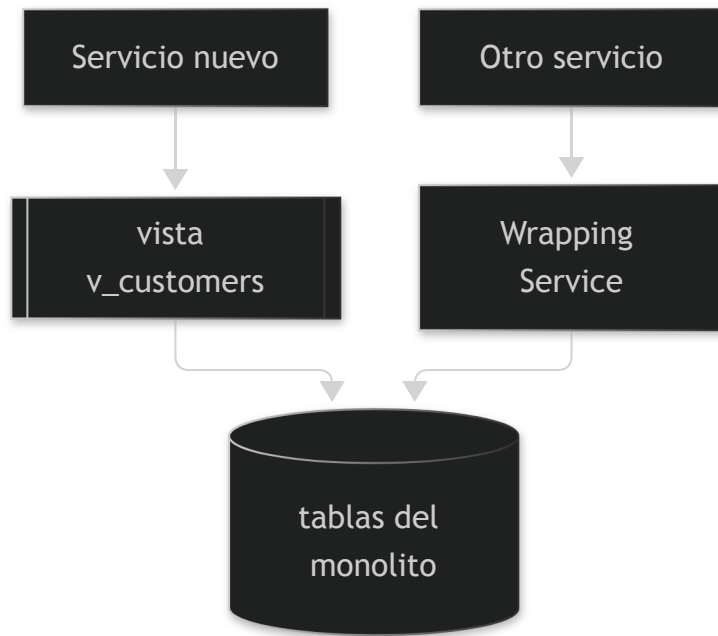
Puentes **temporales** para dejar de acceder a las tablas directamente.

Database View

- Se expone una vista (solo lectura) con lo que el consumidor necesita.
- El monolito puede cambiar las tablas internas mientras mantenga la vista.

Database Wrapping Service

- Un servicio delante de las tablas: todos pasan por su API.
- Aunque el schema sea horrible, al menos ahora **tiene un dueño** y una interfaz.



Split Table

Una tabla que usan dos contextos distintos (items con precio y stock) se parte en dos: cada contexto se queda con **sus** columnas.

1. Identificar qué columnas le pertenecen a quién.
2. Crear la tabla nueva y copiar los datos.
3. Mover las lecturas, después las escrituras.
4. Borrar las columnas viejas.

Si dos contextos **escriben** la misma columna, no hay split posible: primero hay que decidir quién es el dueño.

Es un caso de **expand/contract**: nunca un RENAME in-place, que rompe al lado que todavía no desplegó.

items (antes)

sku · name · price · stock_qty · warehouse



catalog.items

sku · name · price

inventory.stock

sku · stock_qty ·
warehouse

Move Foreign-Key Relationship to Code

Si `orders.customer_id` tiene una FK a `customers`, y `customers` ahora vive en otro servicio, **la FK no puede cruzar bases de datos**.

- La relación se guarda como un **ID** sin constraint.
- La integridad se valida en código: consultando la API del otro servicio, o con eventos (`CustomerDeleted`).
- Los joins pasan a hacerse en la aplicación, o se guarda una **copia local** de lo que se necesita (nombre, email).

Se pierde integridad referencial garantizada por la DB. Hay que decidir qué pasa cuando el cliente se borra: ¿se prohíbe, se anonimiza, se deja huérfana la orden?

```
migration.sql

-- Antes: una sola DB
ALTER TABLE orders
  ADD CONSTRAINT fk_customer
    FOREIGN KEY (customer_id)
      REFERENCES customers(id);

-- Después: orders en su propia DB
ALTER TABLE orders
  DROP CONSTRAINT fk_customer;

-- customer_id queda como dato,
-- validado por el servicio de órdenes
```

Sincronizar durante la transición: el problema del dual write

Mientras monolito y servicio conviven, los dos necesitan los datos. Tentación: **escribir en las dos DBs**.

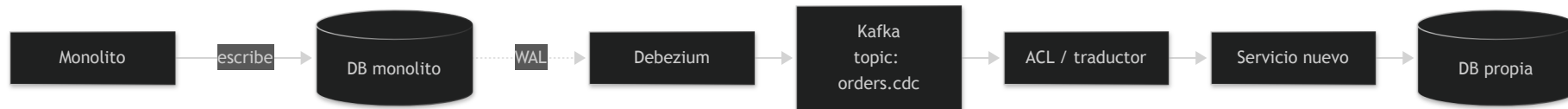
```
save_in_monolith_db(order)
save_in_new_service_db(order)  # ¿y si falla esto?
```

- No hay transacción que cubra las dos.
- Si se cae en el medio, las DBs **divergen** y nadie se entera.
- Es el mismo problema que vieron con DB + broker en `async-patterns-kafka`.

Soluciones:

- **CDC con Debezium + Kafka:** la DB que es fuente de verdad publica sus cambios; la otra los consume.
- **Transactional Outbox:** el evento se escribe en la misma transacción que el cambio.
- **Una sola fuente de verdad** en cada momento: se decide explícitamente quién es dueño, y el otro es una **réplica** de solo lectura.

Sincronización con CDC: el camino típico



1. **Fase 1:** el monolito es dueño; el servicio nuevo tiene una réplica vía CDC y **solo lee**.
2. **Fase 2:** el servicio nuevo pasa a ser dueño; el monolito recibe los cambios de vuelta (CDC u Outbox) mientras alguien lo siga usando.
3. **Fase 3:** nadie lee del monolito; se borra la sincronización y las tablas viejas.

Rollback de datos y cuándo está terminado

Fase	Escribe	Rollback
1	Monolito	Dar vuelta la ruta
2	Servicio nuevo; el monolito recibe copia	Difícil: requiere reconciliar
3	Servicio nuevo; sin sincronización	Ya no hay vuelta barata

- **Un feature flag no revierte datos.** Si la sincronización divergió, volver al monolito es leer datos viejos. La **reconciliación** se define *antes* de entrar en la fase 2.
- **No meter 2PC** para "salvar" el corte: Sagas y consistencia eventual (siguiente slide).

Terminado quiere decir:

- La ruta y el módulo viejos están **borrados**.
- Hay **un solo escritor** por tabla.
- El usuario de DB del monolito **ya no tiene grant** de escritura sobre esa tabla.
- El rollback se **ensayó**, no solo se escribió.

Una convención de "dejamos de escribir" no es un corte. El REVOKE sí.

Adiós ACID: Sagas y consistencia eventual

En el monolito, "crear orden + reservar stock + cobrar" era **una transacción**. Separado, son tres DBs.

- **Saga**: una secuencia de transacciones locales, con **compensaciones** si algo falla.
- **Coreografía** (eventos) vs. **orquestación** (un workflow): lo vieron en sistemas-distribuidos y async-patterns-kafka.
- El negocio tiene que aceptar **estados intermedios** ("orden pendiente de pago").

Preguntas que hay que hacerle al negocio antes de separar:

- ¿Cuánto tiempo puede estar inconsistente este dato?
- ¿Qué pasa si el stock se reservó pero el pago falló?
- ¿Esta invariante tiene que ser **inmediata**? Si sí, quizás esos datos van en el **mismo** servicio.

Reporting y joins entre servicios

El reporte que hacía un JOIN de 6 tablas ahora cruza 4 servicios. Opciones:

- **Composición por API:** un servicio consulta a los demás y arma el resultado. Simple, pero lento y frágil para volúmenes grandes.
- **CQRS / read models:** cada servicio publica eventos; un servicio de reporting mantiene una **vista materializada** optimizada para esa consulta.
- **Data warehouse / lake:** CDC de todas las DBs hacia un almacén analítico. El reporting deja de ser responsabilidad de los servicios operativos.

Regla: no se vuelve a abrir una conexión SQL "de solo lectura" a la DB de otro servicio. Es la DB compartida otra vez, por la puerta de atrás.

Organización y operación

Conway inverso: primero los equipos

La ley de Conway dice que la arquitectura copia la organización. El **Inverse Conway Maneuver** la usa a favor: **diseñar los equipos** con la forma que se quiere para la arquitectura.

Team Topologies (Skelton & Pais):

- **Stream-aligned teams:** dueños de un flujo de negocio de punta a punta (catálogo, checkout).
- **Platform team:** provee CI/CD, observabilidad, infraestructura como un producto interno.
- **Enabling teams:** ayudan a los demás a adoptar prácticas nuevas.

Si 9 servicios los mantiene **un** equipo que coordina todos los deploys, la arquitectura dice "microservicios" pero la organización dice "monolito".

Ganó la organización.

Lo que se complica en operación

Un bug que antes era un stack trace ahora cruza 4 servicios y 2 brokers.

- **Tracing distribuido y correlation IDs:** sin ellos, cada incidente es arqueología (clase 11, Observabilidad).
- **Contract testing:** el monolito y los servicios nuevos cambian a ritmos distintos; los contratos se rompen en silencio (deck testing). El monolito es el **primer consumidor** del contrato.
- **Service mesh:** mTLS, retries, timeouts, mirroring de tráfico sin tocar el código (deck data-plane).
- **Fallas parciales:** timeouts, circuit breakers, degradación elegante. En el monolito, "el otro módulo está caído" no existía.
- **On-call:** cada equipo es dueño de su servicio **también a las 3 de la mañana**.

¿Cómo sé que la migración funciona?

Métricas DORA – medir antes de empezar, y en cada fase:

- **Deploy frequency:** ¿cada equipo despliega más seguido?
- **Lead time for changes:** ¿cuánto tarda un commit en llegar a producción?
- **Change failure rate:** ¿qué porcentaje de deploys causa un incidente?
- **Time to restore (MTTR):** ¿cuánto se tarda en recuperarse?

Métricas propias de la migración:

- **Deploys que requieren coordinar** a más de un equipo: debería tender a cero.
- **% del tráfico** que ya no pasa por el monolito.
- Tablas del monolito que todavía lee otro servicio.
- Por tajada: **delta de error rate** vs. el monolito, **lag del CDC**, y si el rollback se **ensayó**.

Si después de 6 meses el lead time es el mismo y los deploys siguen coordinados, la migración no está logrando su objetivo, aunque haya 10 servicios nuevos.

Anti-patterns de migración

- **Monolito distribuido:** servicios que se despliegan juntos y comparten DB. Lo peor de los dos mundos.
- **DB compartida "por ahora":** el "por ahora" más largo de la industria.
- **Librería compartida con lógica de dominio:** un `common-domain.jar` que todos importan. Cambiar una regla obliga a redespregar a todos.
- **Llamadas sincrónicas encadenadas (chatty):** una request del usuario dispara 12 llamadas en serie; la disponibilidad total es el producto de todas.
- **Big-bang disfrazado:** "migración incremental" que no pone nada en producción durante 8 meses.
- **Extraer sin sacar:** el servicio nuevo existe, pero el código viejo sigue vivo en el monolito "por las dudas". Ahora hay dos que mantener.

Taller grupal

Cómo funciona el taller

Grupos de **3-4 personas**. Una sola hoja (o pizarrón) por grupo.

1. **Leer el caso** (3 min): dos slides con la situación de MegaShop.
2. **Discutir en grupo** (10 min): las preguntas clave, en orden. No hace falta tener todo: vale más una respuesta defendida que cinco a medias.
3. **Puesta en común** (7 min): cada grupo defiende **un** bloque de preguntas (lo asignamos ahora), en 90 segundos. Los demás intervienen solo si llegaron a algo **distinto**.

El producto final es un **plan de migración en una hoja**: qué se extrae primero, cómo, qué pasa con los datos, y cómo se vuelve atrás.

El caso MegaShop (1/2): la situación

MegaShop es un e-commerce de **7 años**. Todo corre en **un solo deployable Java** contra **una base PostgreSQL**. Son **40 devs en 5 equipos**, y sale un release **cada 3 semanas**, un sábado a la mañana, con un deploy **manual** que lleva 4 horas.

El último **Black Friday**, el catálogo recibió 30× el tráfico normal. El proceso se quedó sin memoria y **se cayó el checkout** junto con todo lo demás: 3 horas sin ventas.

Además:

- El equipo de **notificaciones** espera 3 semanas para cambiar un template de mail.
- Un cambio en **catálogo** rompió **pagos** dos veces este año.



El caso MegaShop (2/2): el pedido y las restricciones

El directorio le pidió al CTO: "**pasar todo a microservicios en 6 meses, antes del próximo Black Friday**". El CTO les pide a ustedes un plan.

Lo que encontraron al mirar el código y la base:

- **Checkout** es **una transacción**: crea la orden, descuenta stock y registra el pago.
- orders tiene **FK** a users y a inventory.
- Un **cron nocturno** dentro del monolito recalcula el stock y manda los mails de "volvió a entrar".
- El equipo de **BI** arma sus reportes con **SQL directo** sobre la base de producción.
- Hay tests unitarios, pero **casi ningún test end-to-end**, y los logs quedan en cada servidor.

El pedido

Todo a microservicios, en 6 meses.

El dolor real

Black Friday, releases coordinados, cambios que rompen a otro equipo.

Lo que no se ve en el diagrama

El cron, el SQL de BI, la transacción de checkout, el deploy manual.

Preguntas clave (1/2): decidir y cortar

A. ¿Hay que migrar? ¿Así?

1. ¿Qué problema de negocio resolvería la migración? ¿Es el mismo que pidió el directorio?
2. ¿"Todo en 6 meses" es un plan incremental o un big-bang con otro nombre?
3. ¿Qué **prerrequisitos** faltan antes de extraer el primer servicio?

B. ¿Dónde cortar?

4. ¿Qué **bounded contexts** ven? ¿Coinciden con los módulos actuales?
5. ¿Cuál es el **primer servicio** a extraer, y por qué (acoplamiento, valor, riesgo)?
6. ¿El Black Friday se resuelve extrayendo un servicio, o alcanza con otra cosa?

Preguntas clave (2/2): cómo y con qué datos

C. ¿Cómo extraer?

7. ¿Qué **patrón** usan para el primer servicio (Strangler Fig, Branch by Abstraction, ...)? ¿Dónde va la capa de interceptación?
8. ¿Qué pasa con el **cron nocturno**? ¿A qué servicio le pertenece cada parte?

D. ¿Qué pasa con los datos?

9. ¿Quién es **dueño** de orders, inventory y users? ¿Cómo se sincronizan mientras conviven?
10. Sin la transacción de checkout: ¿qué **estados intermedios** va a ver el usuario, y qué se compensa si falla el pago?
11. ¿Qué hacen con el **SQL directo** de BI?

E. Roadmap: 3 fases, cada una con su **riesgo principal**, **cómo se vuelve atrás** y **una métrica** que diga si funcionó.

Puesta en común

Cada grupo defiende **un** bloque de preguntas, en **90 segundos**:

Bloque	Qué tienen que defender
D. Datos	Dueño de cada tabla, sincronización, qué pasa sin la transacción de checkout
E. Roadmap	Las 3 fases, con riesgo, rollback y métrica
B. Dónde cortar	Contextos y primer servicio
C. Cómo extraer	Patrón y qué hacer con el cron
A. ¿Migrar?	Qué le responden al directorio

Los demás grupos: **solo** si llegaron a algo distinto.

Cierre

Ideas clave

- **Migrar es una decisión, no un destino.** Primero el problema de negocio, después la arquitectura. El monolito modular es una respuesta válida.
- **Las fronteras se descubren:** DDD para el mapa, dependencias, historial de git y datos de producción para contrastarlo.
- **Incremental y reversible:** Strangler Fig, Branch by Abstraction, Parallel Run, toggles. Si el paso no se puede revertir, es demasiado grande.
- **Los datos son la parte difícil:** un dueño por dato, nada de DB compartida, CDC/Outbox para sincronizar, Sagas donde se pierde ACID.
- **La organización manda:** Conway inverso, equipos dueños de punta a punta, y métricas DORA para saber si funciona.
- **Terminado = ruta vieja borrada, un solo escritor, grant revocado, rollback ensayado.**

Para seguir leyendo

- Sam Newman, ***Monolith to Microservices*** (O'Reilly, 2019). Los capítulos 3 y 4 son la columna de esta clase.
- Martin Fowler, *StranglerFigApplication*, *MonolithFirst* y *MicroservicePrerequisites*.
- Chris Richardson, *microservices.io* – patrones de descomposición y de datos.
- Matthew Skelton & Manuel Pais, ***Team Topologies*** (IT Revolution, 2019).
- Adam Tornhill, ***Your Code as a Crime Scene*** – análisis del historial de git.

Fin!!